DAA Assignment – Set 1

Question 1 – Divide and Conquer

You are given an unsorted array A[0…n−1] of n distinct integers. A rotation point is an index i where A[i] > A[i+1]. The array may have zero, one, or more rotation points. Design a divide-and-conquer algorithm to find the number of rotation points in the array.

- Tasks
- Design a divide-and-conquer algorithm that divides the array into left and right halves.
- Implement the algorithm in a programming language of your choice.
- Show how the problem size is reduced in each recursive step.
- Derive the recurrence relation for the running time.
- Solve the recurrence and determine the asymptotic time complexity.
- Compare the result with a simple linear scan O(n) algorithm.

Question 2 – Greedy Method

You are given n coins with denominations d_1, d_2, …, d_n (not necessarily the standard {1, 5, 10, 25}). Given an amount M, design a greedy algorithm to find the minimum number of coins needed to make the amount M. Assume an unlimited supply of each coin denomination.

- Tasks
- Propose a greedy strategy and justify why it works for certain coin systems.
- Implement the algorithm.
- Discuss cases where the greedy strategy may not yield the optimal solution (e.g., coins {1, 3, 4} for amount 6).
- Use an appropriate data structure to efficiently track coins.
- Analyze the time complexity of the algorithm.
- Test the algorithm on different coin systems and amounts, reporting results.

Question 3 – Dynamic Programming

You are given a set of n items, each with a weight w_i and a value v_i, and a knapsack of capacity W. Design a dynamic programming algorithm to determine the

maximum value that can be obtained by selecting items such that the total weight does not exceed W. Each item can be selected at most once (0/1 knapsack problem).

- Tasks
- Identify the optimal substructure and derive the recurrence relation.
- Construct the dynamic programming table.
- Implement the algorithm to compute the maximum value.
- Extend the implementation to print the items selected in the optimal solution.
- Analyze the time and space complexity of the algorithm.
- Experimentally evaluate the algorithm for different values of n and W, and report the results.